Tabla de contenidos

1. *Server-side App*
    a. Flask
        i. Ejemplo básico
        ii. Ejemplo Avanzado
            1. Caso 1: *endpoint* dinámico
            2. Caso 2: argumentos en la URL
            3. Caso 3: datos en el *body*
            4. Caso 4: manejo de errores y respuestas personalizadas

Server-side App

La misión principal del servidor es disponer el contenido para que pueda ser consultado mediante un *web service*. La aplicación que corre en el servidor es la encargada de la lógica e interacción entre cliente-servidor. La información que viaja entre un cliente y un servidor permite generar comunicación entre aplicaciones.

Una aplicación puede estar desarrollada en cualquier lenguaje de programación que permita exponer una API para ser consumida por otras aplicaciones a través de la web. Por ejemplo, podemos tener una aplicación corriendo en Java, y desde nuestro código en Python acceder a esa API.

En Python existe WSGI para exponer APIs. Tambien existen varios *frameworks* de programación que facilitan esta misma tarea, como **Flask** y **Django**. Además, puedes montar tus aplicaciones en servicios o servidores ya disponibles en la web, provistos como Platform-as-a-Service (PaaS) o Infrastructure-as-a-Service (IaaS). Por ejemplo, puedes usar **Heroku** (PaaS), **Digital Ocean** (IaaS), o **Microsoft Azure** (PaaS) para disponer tus APIs en una red pública con alta disponibilidad.

A modo de ejemplo, tenemos una pequeña API utilizando WSGI para levantar una aplicación que responde algunos mensajes. Este código **NO utiliza frameworks como Flask o Django** para implementar los distintos endpoints, el manejo de consultas y la generación de respuestas, por lo que es distinto a lo que se verá en el resto del *notebook*.

Antes de empezar, debes ejecutar el archivo `servidor-API.py` presente en la carpeta `codigo/ejemplo_sin_flask` en la terminal. Luego, puedes ejecutar las siguientes celdas donde utilizaremos `requests` para realizar solicitudes a esta API.

```
import requests

# La URL base de la API creada
BASE_URL = "http://localhost:4444/"

# Podemos consultar a esta ruta
respuesta = requests.get(BASE_URL)
respuesta.status_code
```

```
200
```

```
respuesta.json()
```

```
{'mensaje': 'Hello World'}
```

```
# ¡Vamos a despedirnos haciendo una consulta a otro endpoint de
nuestra API!
respuesta = requests.get(BASE_URL + "goodbye/")
print(respuesta.status_code, respuesta.json())
```

```
200 {'mensaje': 'Que la fuerza esté contigo'}
```

Flask

En esta sección veremos, desde el punto de vista del servidor, cómo crear una API que podrá ser consumida mediante `requests`. En Python, la librería `flask` nos permite levantar una API y asociar diferentes funciones a los métodos (`GET`, `POST`, `PUT`, `DELETE`, `PATCH`) y los *endpoints* deseados.

Para instalar la librería `flask`, en cualquier terminal debes correr el comando `python3 -m pip install flask`. Otra opción, es ejecutar la siguiente celda, que instalará la librería en el mismo entorno en que estés corriendo este jupyter (de todas maneras recomendamos instalarlo desde la consola).

Recuerda que si `python3` no funciona, probar con `python`, `py` o `py3`.

```
!python3 -m pip install flask
```

```
Requirement already satisfied: flask in
/home/jimmy/Documents/Avanzada/serializacion y
networking/.venv/lib/python3.12/site-packages (3.1.3)
Requirement already satisfied: blinker>=1.9.0 in
/home/jimmy/Documents/Avanzada/serializacion y
networking/.venv/lib/python3.12/site-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in
/home/jimmy/Documents/Avanzada/serializacion y
networking/.venv/lib/python3.12/site-packages (from flask) (8.3.3)
Requirement already satisfied: itsdangerous>=2.2.0 in
/home/jimmy/Documents/Avanzada/serializacion y
networking/.venv/lib/python3.12/site-packages (from flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in
/home/jimmy/Documents/Avanzada/serializacion y
networking/.venv/lib/python3.12/site-packages (from flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in
/home/jimmy/Documents/Avanzada/serializacion y
networking/.venv/lib/python3.12/site-packages (from flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in
/home/jimmy/Documents/Avanzada/serializacion y
networking/.venv/lib/python3.12/site-packages (from flask) (3.1.8)
```

Ejemplo básico

La sintaxis básica para levantar una aplicación con flask es:

```python
from flask import Flask, request
import random

app = Flask(__name__)


@app.route("/", methods=["GET"])
def hello_world():
    return {"texto": "Hello, World!"}


@app.route("/numero_aleatorio", methods=["GET", "POST"])
def numero_aleatorio():
    if request.method == "POST":
        numero = random.randint(0, 6)
        return {"texto": f"Tu número es: {numero}", "método": "POST"}

    numero = random.randint(-4444, -11)
    return {"texto": f"Tu número es: {numero}", "método": "GET"}


if __name__ == "__main__":
    app.run(host="localhost", port=4444)
```

1. Primero hacemos `import` de la librería.
2. Creamos una instancia de la aplicación mediante `Flask(__name__)`. El uso de `__name__` es para que la aplicación tenga el nombre del archivo que contiene la app.
3. Para definir los distintos *endpoints* de la API:
   - Usamos `@app.route` para definir el *endpoint* y el o los métodos que permiten acceder a la función.
   - Luego, definimos la función que se va a ejecutar y retornaremos la respuesta a la consulta.
   - Adentro de la función, podemos usar `request.method` para identificar el método utilizado para llamar a dicha función.
4. Dentro del `__name__ == "__main__"` hacemos `app.run` para ejecutar el *script*.

Finalmente, en este curso disponemos de 2 opciones para ejecutar una aplicación con `Flask`:

1. Usar el comando `python` para ejecutar el archivo, por ejemplo, `python3 servidor_1.py`
2. Usar el comando `flask` y `run` para levantar la API, por ejemplo, `flask --app servidor_1 run --port 4444`. Si no se indica el puerto, en esta opción se utilizará el puerto por defecto de flask, el 5000.

En ambos casos, aparecerá el siguiente mensaje:

WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.

¡No te preocupes! Esto es porque cuando uno quiere levantar una API para el público, se utilizan WSGI para ejecutar y exponer una API creada por `Flask`.

Probando la API

La siguiente celda inicia el servidor directamente desde el notebook (sin necesidad de abrir una terminal). Una vez iniciado, dispondremos los siguientes *endpoints*:

- (`GET`) `http://localhost:4444/`: ejecutará la función `hello_world`.
- (`GET`) `http://localhost:4444/numero_aleatorio`: retornará un número aleatorio entre **-4444 y -11**.
- (`POST`) `http://localhost:4444/numero_aleatorio`: retornará un número aleatorio entre **0 y 6**.

```
import threading
import random
import logging
from werkzeug.serving import make_server
from flask import Flask
# Nota: 'request' es el objeto de Flask (singular),
# distinto de la librería de HTTP 'requests' (plural)
from flask import request as flask_req

logging.getLogger("werkzeug").setLevel(logging.ERROR)  # suprimir logs
de werkzeug

app_1 = Flask("servidor_1")

@app_1.route("/", methods=["GET"])
def hello_world():
    return {"texto": "Hello, World!"}

@app_1.route("/numero_aleatorio", methods=["GET", "POST"])
def numero_aleatorio():
    if flask_req.method == "POST":
        numero = random.randint(0, 6)
        return {"texto": f"Tu número es: {numero}", "método": "POST"}
    numero = random.randint(-4444, -11)
    return {"texto": f"Tu número es: {numero}", "método": "GET"}

servidor_1 = make_server("localhost", 4444, app_1)
threading.Thread(target=servidor_1.serve_forever, daemon=True).start()
print("Servidor 1 iniciado en http://localhost:4444")
```

```
Servidor 1 iniciado en http://localhost:4444
```

```
import requests
```

```
respuesta = requests.get("http://localhost:4444/")
respuesta.json()
```

```
{'texto': 'Hello, World!'}
```

```
respuesta = requests.get("http://localhost:4444/numero_aleatorio")
respuesta.json()
```

```
{'método': 'GET', 'texto': 'Tu número es: -1409'}
```

```
respuesta = requests.post("http://localhost:4444/numero_aleatorio")
respuesta.json()
```

```
{'método': 'POST', 'texto': 'Tu número es: 6'}
```

```
servidor_1.shutdown()
print("Servidor 1 detenido")
```

```
Servidor 1 detenido
```

Ejemplo Avanzado

Cuando consumimos una API, a veces necesitamos o queremos:

- Que algún *endpoint* cambie dinámicamente.
- Obtener los argumentos entregados en la consulta.
- Enviar datos como argumentos y/o dentro del *body*.
- Manejar errores y/o tener un mayor control de las respuestas que se mandan (códigos de respuesta, tipo de contenido).

Vamos a ver un ejemplo y entender cada función:

```
import json
from flask import Flask, Response, request, abort

app = Flask(__name__)


# Caso 1: endpoint dinámico
@app.route("/hi/<string:username>", methods=["GET"])
def saludar(username):
    return {"texto": f"Hola {username}!"}


# Caso 2: argumentos en la URL
@app.route("/argumentos", methods=["GET"])
def argumentos():
    nombre = request.args.get("nombre", default="Guillermo", type=str)
    especie = request.args.get("especie", default="Gato", type=str)
    edad = request.args.get("edad", default=0, type=int)
    return {"texto": f"Hola {especie} {nombre} de {edad} años!"}
```

```
# Caso 3: datos en el body
@app.route("/body", methods=["POST"])
def datos_en_body():
    body_data = request.get_json(force=True)
    numero_1 = body_data["var_1"]
    numero_2 = body_data["var_2"]
    resultado = numero_1 + numero_2
    return {"var_1": numero_1, "var_2": numero_2, "result": resultado}


# Caso 4: manejar errores y/o tener más control de las respuestas que
se entregan
@app.route("/error/<int:instruccion>", methods=["DELETE"])
def error(instruccion):
    if instruccion == 0:
        abort(400)
    if instruccion == 1:
        return Response(status=418)
    if instruccion == 2:
        return Response(status=200, response='Podemos mandar textos',
content_type='text/plain')
    if instruccion == 3:
        respuesta = json.dumps({'mensaje': 'También JSONs'})
        return Response(status=202, response=respuesta,
content_type='application/json')



if __name__ == "__main__":
    app.run(host="localhost", port=4444)
```

La siguiente celda inicia este servidor directamente desde el notebook.

```
import json
import threading
import logging
from werkzeug.serving import make_server
from flask import Flask, Response, abort
from flask import request as flask_req

logging.getLogger("werkzeug").setLevel(logging.ERROR)

app_2 = Flask("servidor_2")

@app_2.route("/hi/<string:username>", methods=["GET"])
def saludar(username):
    return {"texto": f"Hola {username}!"}
```

```
@app_2.route("/argumentos", methods=["GET"])
def argumentos():
    nombre = flask_req.args.get("nombre", default="Guillermo",
type=str)
    especie = flask_req.args.get("especie", default="Gato", type=str)
    edad   = flask_req.args.get("edad",   default=0,
type=int)
    return {"texto": f"Hola {especie} {nombre} de {edad} años!"}

@app_2.route("/body", methods=["POST"])
def datos_en_body():
    body_data = flask_req.get_json(force=True)
    numero_1 = body_data["var_1"]
    numero_2 = body_data["var_2"]
    return {"var_1": numero_1, "var_2": numero_2, "result": numero_1 +
numero_2}

@app_2.route("/error/<int:instruccion>", methods=["DELETE"])
def error(instruccion):
    if instruccion == 0:
        abort(400)
    if instruccion == 1:
        return Response(status=418)
    if instruccion == 2:
        return Response(status=200, response="Podemos mandar textos",
content_type="text/plain")
    if instruccion == 3:
        resp = json.dumps({"mensaje": "También JSONs"})
        return Response(status=202, response=resp,
content_type="application/json")

servidor_2 = make_server("localhost", 4444, app_2)
threading.Thread(target=servidor_2.serve_forever, daemon=True).start()
print("Servidor 2 iniciado en http://localhost:4444")
```

```
Servidor 2 iniciado en http://localhost:4444
```

Caso 1: *endpoint* dinámico

```
@app.route("/hi/<string:username>", methods=["GET"])
def saludar(username):
    return {"texto": f"Hola {username}!"}
```

En este caso, cuando se hace `"<string:username>"` dentro del *endpoint*, se interpreta que dicha sección de la URL puede ir mutando. Por ejemplo, hacer `"/hi/Luna"`, `"/hi/Cachirulo"`, `"/hi/Guillermo"` harán *match* con este *endpoint* y ejecutarán la función `saludar`.

A continuación vamos a probar este *endpoint* con diferentes nombres:

```
import requests

respuesta = requests.get("http://localhost:4444/hi/Luna")
respuesta.json()
```

```
{'texto': 'Hola Luna!'}
```

```
respuesta = requests.get("http://localhost:4444/hi/Cachirulo")
respuesta.json()
```

```
{'texto': 'Hola Cachirulo!'}
```

```
respuesta = requests.get("http://localhost:4444/hi/Guillermo")
respuesta.json()
```

```
{'texto': 'Hola Guillermo!'}
```

Caso 2: argumentos en la URL

```
@app.route("/argumentos", methods=["GET"])
def argumentos():
    nombre = request.args.get("nombre", default="Guillermo", type=str)
    especie = request.args.get("especie", default="Gato", type=str)
    edad = request.args.get("edad", default=0, type=int)
    return {"texto": f"Hola {especie} {nombre} de {edad} años!"}
```

En este caso, cuando agregamos argumentos en la URL (por ejemplo `"/args?name=Lucky"`) se debe utilizar `request.args` para acceder a dicha información. En particular, como `request.args` es un diccionario, usamos el método `.get(key, valor_por_defecto, tipo_de_dato)` para obtener el argumento cuya llave sea `key`, transformarlo al `tipo_de_dato` entregado, y -en caso de no existir dicho argumento- usamos el valor por defecto.

A continuación vamos a probar este *endpoint* con diferentes argumentos:

```
respuesta = requests.get("http://localhost:4444/argumentos?
nombre=Pepa&especie=Tortuga&edad=70")
respuesta.json()
```

```
{'texto': 'Hola Tortuga Pepa de 70 años!'}
```

```
respuesta = requests.get("http://localhost:4444/argumentos?
nombre=Cachirulo&edad=5")
respuesta.json()
```

```
{'texto': 'Hola Gato Cachirulo de 5 años!'}
```

```
respuesta = requests.get("http://localhost:4444/argumentos")
respuesta.json()
```

```
{'texto': 'Hola Gato Guillermo de 0 años!'}
```

```
parametros = {"nombre": "Luna", "edad": 8, "informacion_extra": "Le
gusta el churu"}

respuesta = requests.get("http://localhost:4444/argumentos",
params=parametros)
respuesta.json()
```

```
{'texto': 'Hola Gato Luna de 8 años!'}
```

Caso 3: datos en el *body*

```
@app.route("/body", methods=["POST"])
def datos_en_body():
    body_data = request.get_json(force=True)
    numero_1 = body_data["var_1"]
    numero_2 = body_data["var_2"]
    resultado = numero_1 + numero_2
    return {"var_1": numero_1, "var_2": numero_2, "result": resultado}
```

En este caso, cuando agregamos datos en el *body*, existen varias formas de tratar con esta información, pero una de estas es usar `request.get_json(force=True)` para acceder a dicha información. En particular, usamos `force=True` para obligar que cualquier información enviada en `data` se transforme en un JSON.

A continuación vamos a probar este *endpoint* con diferentes argumentos:

```
import json

body = {
    'var_1': 11,
    'var_2': 13
}

respuesta = requests.post("http://localhost:4444/body",
data=json.dumps(body))
respuesta.json()
```

```
{'result': 24, 'var_1': 11, 'var_2': 13}
```

```
body = {
    'var_1': 4444,
    'var_2': 0
}

respuesta = requests.post("http://localhost:4444/body",
data=json.dumps(body))
respuesta.json()
```

```
{'result': 4444, 'var_1': 4444, 'var_2': 0}
```

Caso 4: manejo de errores y respuestas personalizadas

```
@app.route("/error/<int:instruccion>", methods=["DELETE"])
def error(instruccion):
    if instruccion == 0:
        abort(400)
    if instruccion == 1:
        return Response(status=418)
    if instruccion == 2:
        return Response(status=200, response='Podemos mandar textos',
content_type='text/plain')
    if instruccion == 3:
        respuesta = json.dumps({'mensaje': 'También JSONs'})
        return Response(status=202, response=respuesta,
content_type='application/json')
```

En este caso vemos distintas forma de generar respuestas a nuestras consulta:

- `abort(codigo)` recibe un código de estado y -al igual que un `raise`- levanta una excepción (es específico, `HTTPException`) que termina la ejecución de la función y envía una respuesta con el código recibido.
- `Response()` corresponde al objeto que es enviado como respuesta a una consulta. Por lo general, cuando el `return` de una función recibe algo distinto a una `Response`, internamente se ejecuta un método que se encarga de transformar lo retornado en una `Response` (el método se llama `make_response()`).

  El objeto `Response` puede recibir los siguientes **parámetros opcionales**:

  - `status`: Código de estado asociado a la respuesta. Si no se entrega se utiliza 200.
  - `response`: Contenido del body asociado a la respuesta. Si no se entrega, significa que el body de la respuesta está vacío.
  - `content_type`: Tipo de dato del body. Si no se entrega, lo definirá de forma automática.

```
respuesta = requests.delete("http://localhost:4444/error/0")
respuesta
```

```
<Response [400]>
```

```
respuesta = requests.delete("http://localhost:4444/error/1")
respuesta
```

```
<Response [418]>
```

```
respuesta = requests.delete("http://localhost:4444/error/2")
print(respuesta)
print(respuesta.text)
```

```
<Response [200]>
Podemos mandar textos
```

```
respuesta = requests.delete("http://localhost:4444/error/3")
print(respuesta)
print(respuesta.json())
```

```
<Response [202]>
{'mensaje': 'También JSONs'}
```

```
servidor_2.shutdown()
print("Servidor 2 detenido")
```

```
Servidor 2 detenido
```